

CSC1029 OO Programming

Lecture 5 – defining and using objects





Reminder – In week 6 – we have an assessment!

Remember to continue with the practical exercises.





What's to come today?

- what is an object-oriented program (OOP)?
 - what are objects and how are they described?
 - creating objects in Java
 - example – String objects
 - example – objects for basic input and output
 - object definition
- 



What is an Object-Oriented Program?

An implementation of an **algorithm**

Aimed at solving a particular problem

Consists of a collection of interacting **objects**

So, what are objects?



What is an Object?

A software representation of

A real world entity ...

... for example, a book



Can be an abstract concept ...

... for example, a bank account



How are Objects Described?

In terms of what they look like ...

State:

In terms of what can be done with them ...

Behaviour:



Example – Using String Objects ⁽¹⁾

A String is a non-primitive type representing a sequence of characters

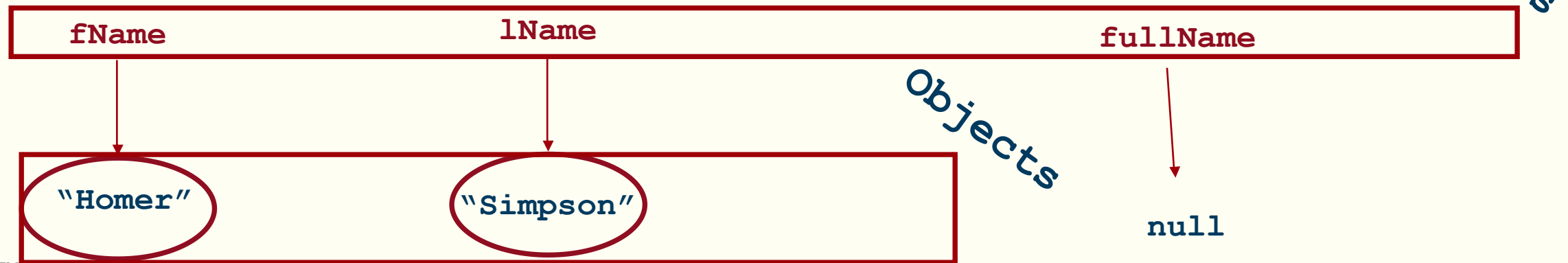
```
class MyProgram {  
    public static void main(String args[]){  
        String fName = "Homer";  
        String lName = new String("Simpson");  
    }  
}
```

Two String objects are created: referenced by identifiers fName and lName

Example – Using String Objects ⁽²⁾

The process of creating an object is called instantiation.

```
String fName = "Homer"; // assignment to a literal
String lName = new String("Simpson"); // using new
String fullName; // not referencing any object
```



Example – Using String Objects ⁽³⁾

Objects are described in terms of **state** & **behaviour** ...

↓
Relates to **method**
calls made via
the object reference

for example

```
int len = fName.length();  
char initial = fName.charAt(0);
```

fName



↓
Relates to the
Char data managed
within the object

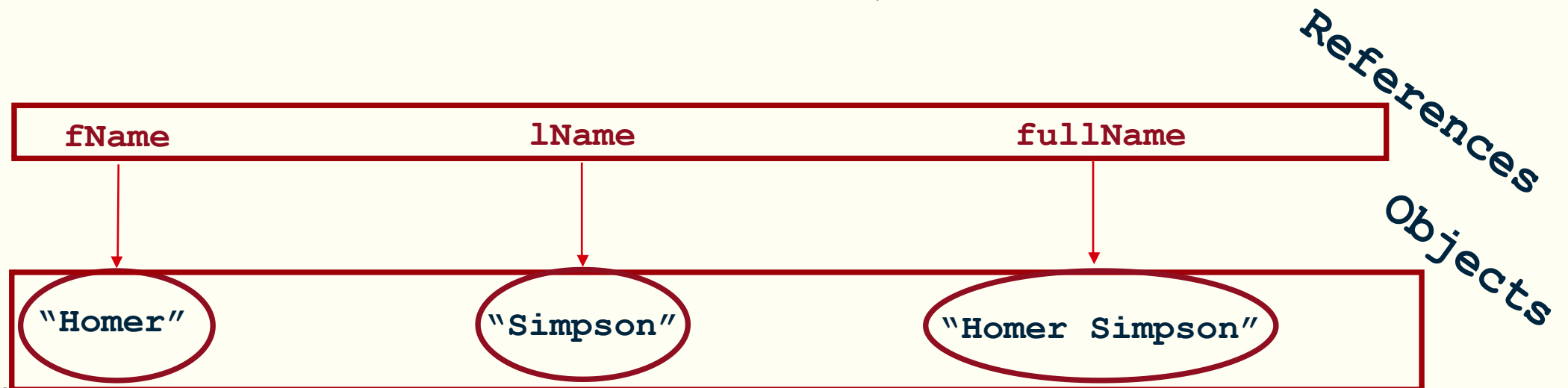
→ len will be 5
initial will be 'H'

Example – Using String Objects ⁽⁴⁾

Strings may be concatenated ...

```
String fName = "Homer"; // assignment to a literal  
String lName = new String("Simpson"); // using new  
String fullName; // not referencing any object
```

```
fullName = fName + " " + lName;
```



Code Example ⁽¹⁾



Strings.java

Example – Objects for Console Output

```
String fName = "Homer"; // assignment to a literal
String lName = new String("Simpson"); // using new
String fullName; // not referencing any object
fullName = fName + " " + lName;

System.out.println("First name is: " + fName);

System.out.println("Last name is: " + lName);

System.out.println("Full name is: " + lName + ", " + fName);
```

Console

```
First name is: Homer
Last name is: Simpson
Full name is: Simpson, Homer
```

Example



Output.java

Example – Objects for Console Input

```
Scanner input = new Scanner(System.in); // create a Scanner
System.out.print("Enter first name: "); // output message
String fName = input.nextLine(); // read from console
System.out.print("Enter last name: "); // output message
String lName = input.nextLine(); // read from console
```

```
System.out.println("Full name is: " + lName + ", " + fName);
```

Console

```
Enter first name: Homer
Enter Last name:  Simpson
Full name is: Simpson, Homer
```

Code Example (2)



Input.java

How are Objects Described? revisited

In terms of what they look like ...

State:

In terms of what can be done with them ...

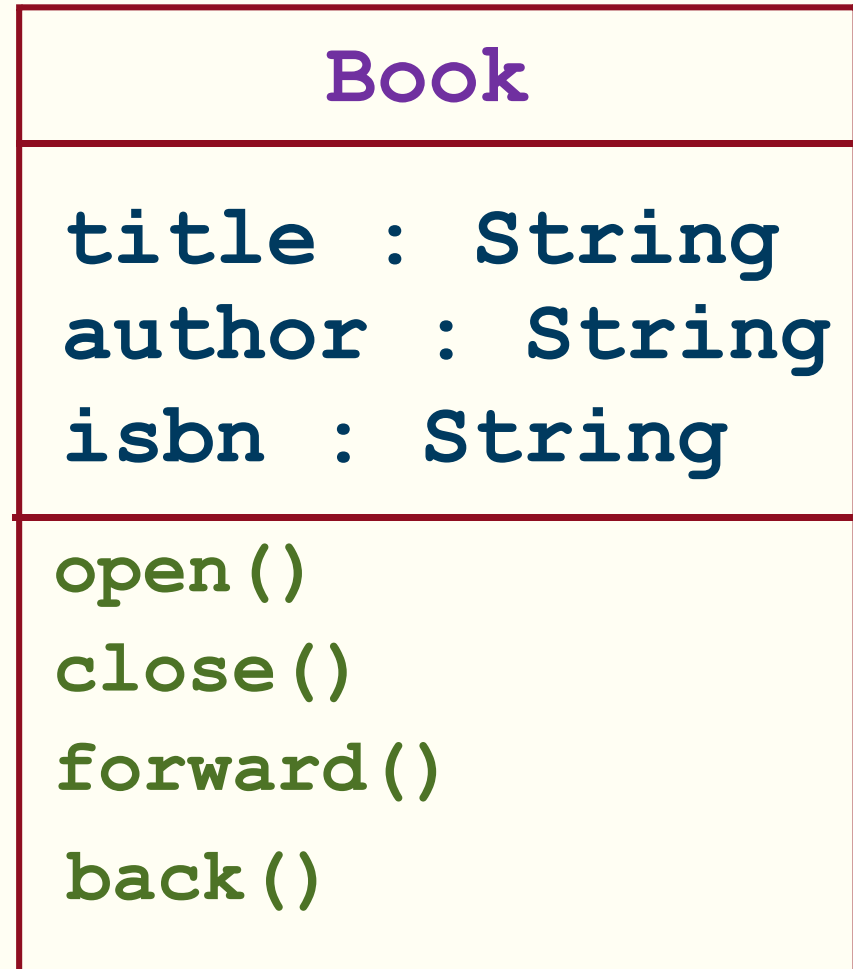
Behaviour:



Using Classes to Describe Objects

A **class** is used to describe the **state** and **behaviour** of objects of the same type

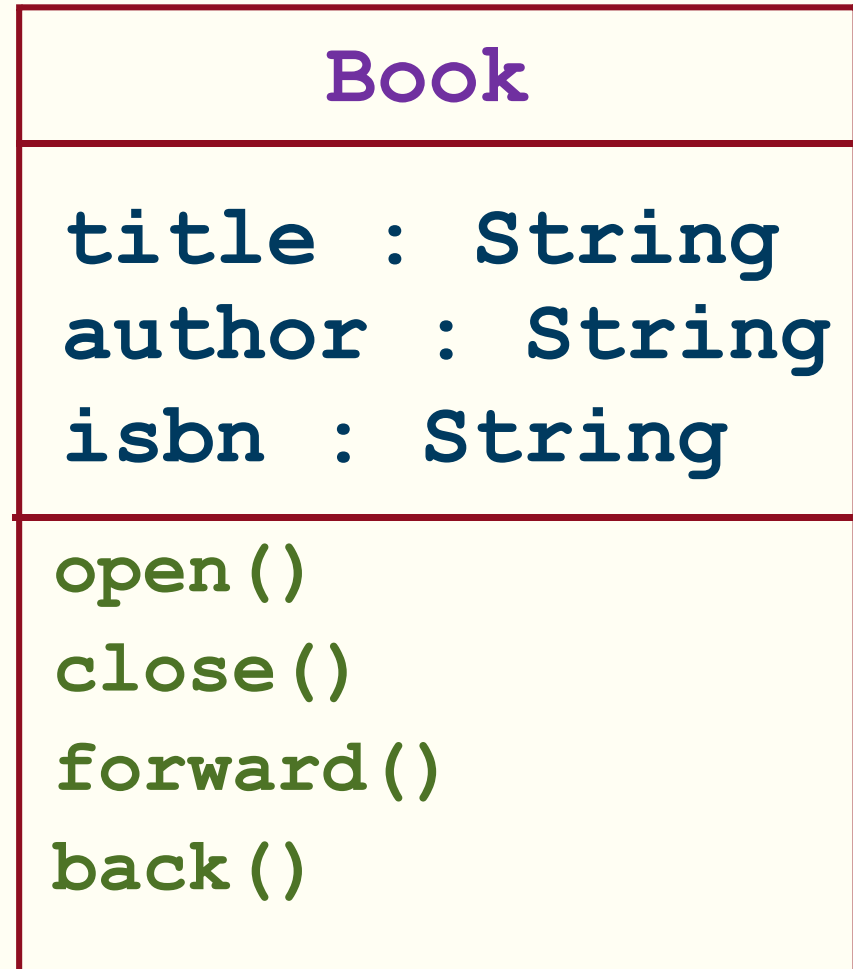
A class is like a 'blueprint' for an object



Using Classes to Describe Objects ⁽¹⁾

Instance variables
describe the state
of an object

Methods are used to
describe object
behaviour



Using Classes to Describe Objects (2)



Defining Classes in Java

Book

title : String
author : String
isbn : String

open()
close()
forward()
back()

```
public class Book {  
  
    public String title;  
    public String author;  
    public String isbn;  
  
    public void open() { ... }  
    public void close() { ... }  
    public void forward() { ... }  
    public void close() { ... }  
}
```

A Car Object Example

Any real life
item, thing
or person

They all have
state and
behaviours

generally nouns

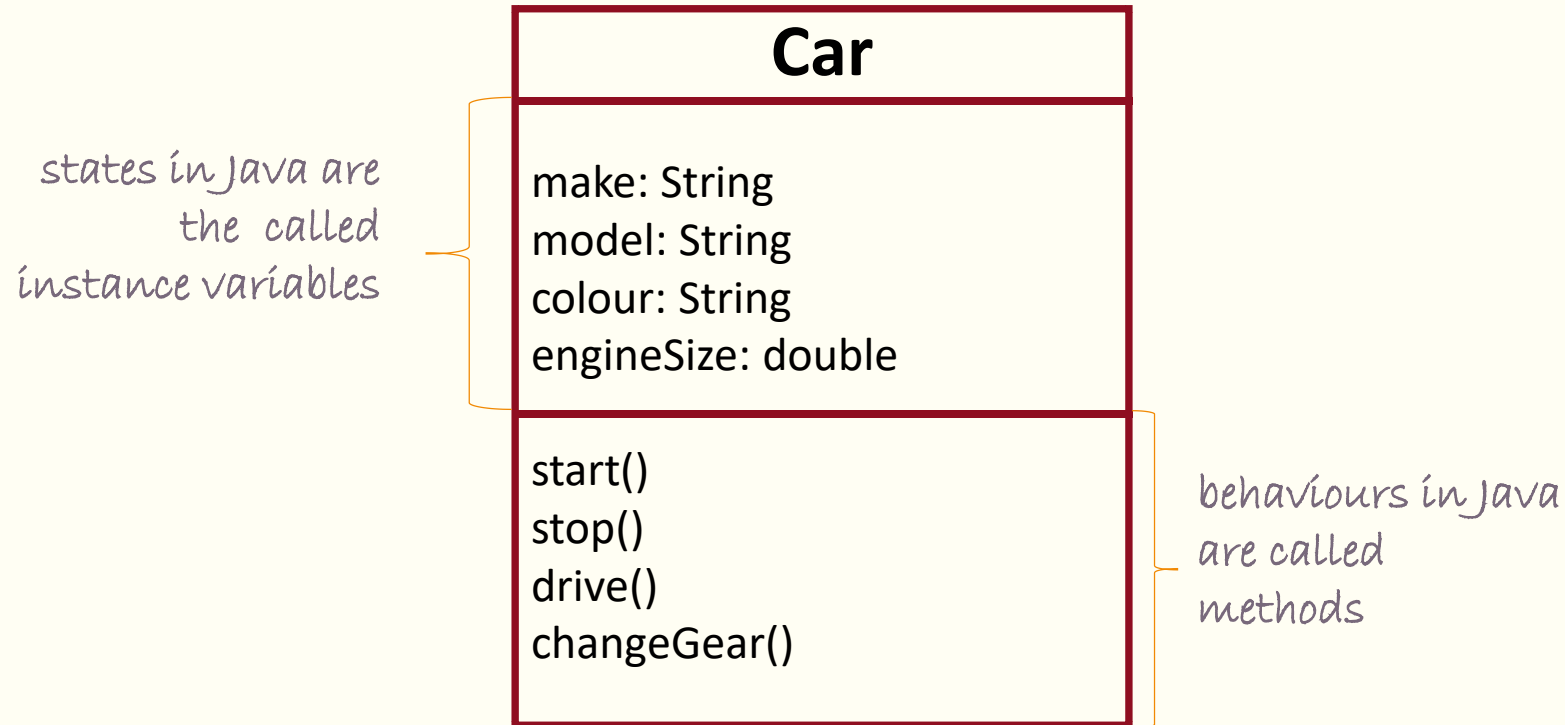


States: make, model, colour, engine size

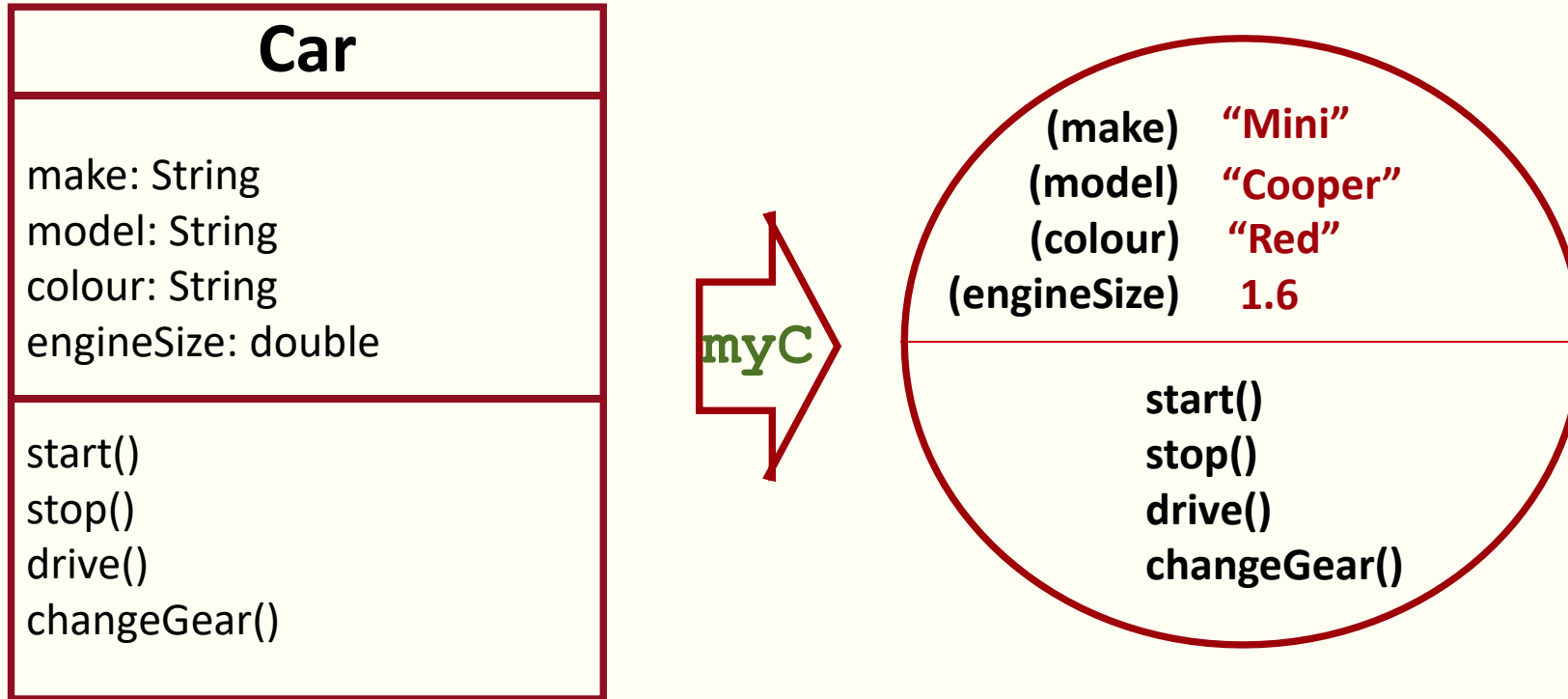
Behaviours: start, stop, drive,
changeGear

generally verbs

Basic Outline of a Car Class



Using a Class – Creating an Object



```
Car myC = new Car() ; // an instance of Car
myC.make = "Mini" ; // set make
myC.model = "Cooper" ; // set model
myC.colour = "Red" ; // set colour
myC.engineSize = 1.6 ; // set engineSize
```

How to Code the Car Class

Object class

A class for the type of object you are programming e.g. a car

Tester class

A tester class to test the functionality of the object. This is the class that NEEDS a main() method. The main() method will create a variable of type car (in this instance). You will then test all of the methods in the object class.

Step 1 – Write Your Object Class

```
public class Car {  
    public String make;  
    public String model;  
    public String colour;  
    public double engineSize;  
  
    public void drive() {  
        System.out.println("I am driving");  
    }  
}
```

instance variables

method

Step 2 – Write Your Tester Class ⁽¹⁾

```
public class CarTester {  
    public static void main (String[] args) {  
        //create a variable of type car  
        //set the make  
        //set the model  
        //set the colour  
        //set the engineSize  
        //Code to test the drive method  
    }  
}
```

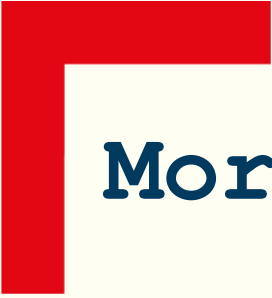
Step 2 – Write Your Tester Class ⁽²⁾

```
public class CarTester {  
    public static void main (String[] args) {  
  
        Car myC = new Car(); // create a Car object  
        myC.make = "Mini";   // set the make  
        myC.model = "Cooper"; // set the model  
        myC.colour = "Red";  // set the colour  
        myC.engineSize = 1.6; // set the engineSize  
        myC.drive();         // use drive method  
  
    }  
}
```

Code Example



- **Car.java**
- **CarTester.java**

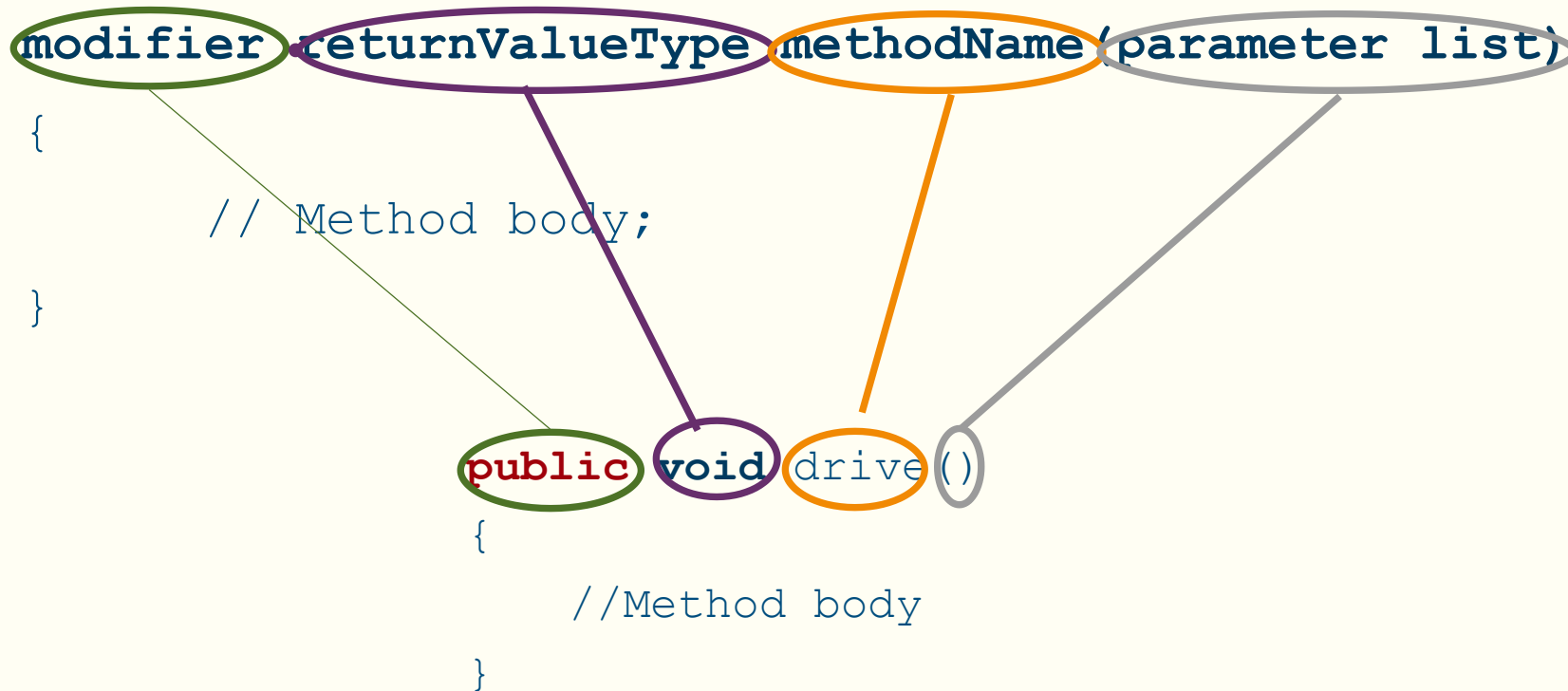


More Detail on Methods

- The *behaviour* of a real world object is described using *methods* in Java.
- An object *must exist* before a behaviour (*instance*) method can be used:
e.g. `myC.drive()` ; // `myC` is an object name
- A method defined using *static* is not associated with an object.
- Some methods that we have seen:
`public static void main (String [] args)`
`public void drive()`

Methods in Java

- Each method consists of a method header and a method body.

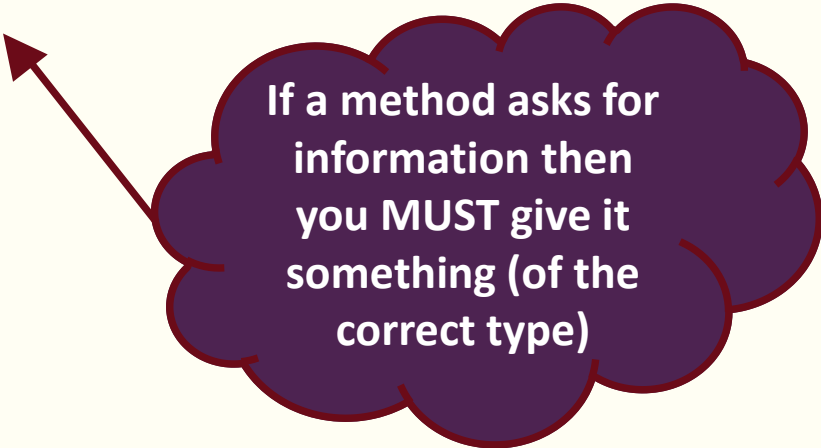


Why Would We Need a Parameter List?

- Because that method requires information to behave properly.
 - *Every car is going to have a different make so we could pass this information to a method*

```
public void setMake(String newMake) {  
    make = newMake;  
}
```

We will return to discuss why this is better programming rather than setting the variable directly in the tester.



If a method asks for information then you **MUST** give it something (of the correct type)

Methods and Return Types

```
public String getColour()  
{  
    return colour;  
}
```

Whatever you
say you are
going to give
back you better
give back

```
public void drive()  
{  
    //code for driving  
}
```



Instantiation & Object References ⁽¹⁾

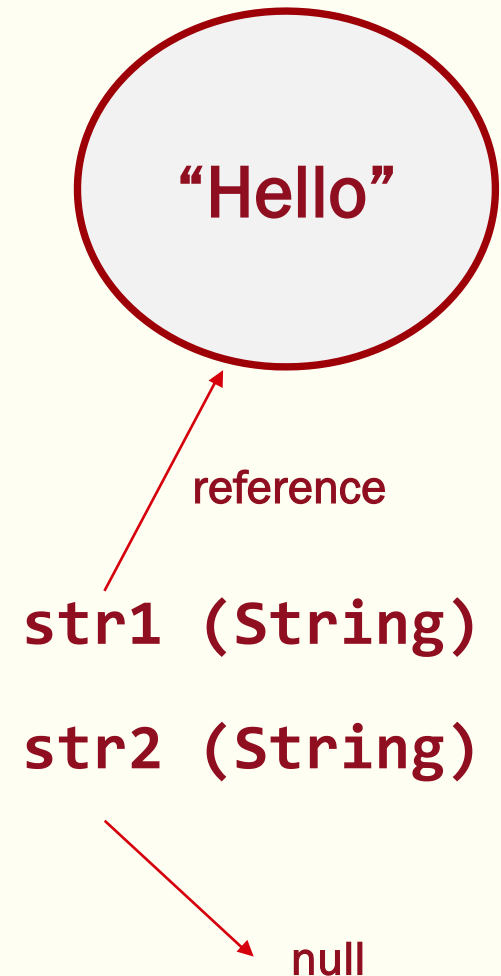
Java objects reside in an area called the **heap (heap memory)**.

The **heap** is created when the JVM starts up and can change in size as an application executes.

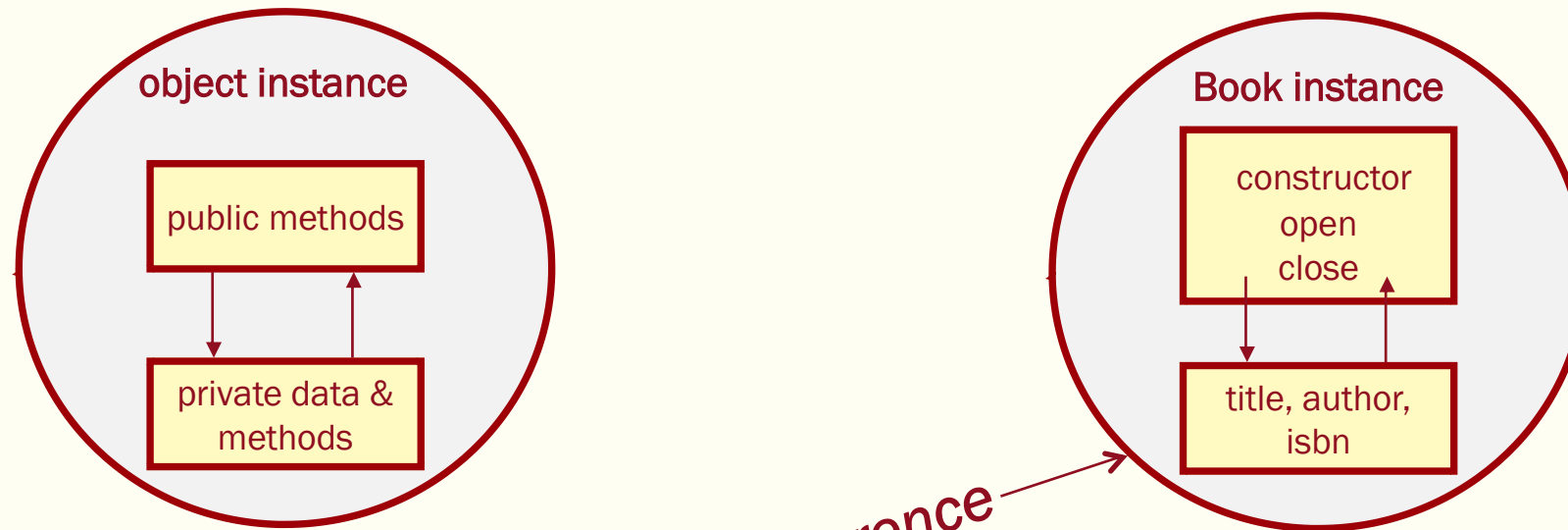
When the **heap** becomes full, garbage (unreferenced objects) is collected, making space for new objects.

An object's identifier (variable name) is a reference to (or memory location of) its associated object

.... Either null (referencing nothing) or not null



Instantiation & Object References (2)



object reference →

Book mybk = new Book("Java for CSC1027", "J. Bloggs", "1234567");

Exercise – Fill in States and Behaviours



Student



What's Next?

Class Design Essentials:

- a brief introduction to UML
 - standards for Java implementation
 - more examples!
- 